

Machinator Outline

April 6, 2026

Preface

This outline is a working reference for the Machinator project. Its point is to compress a wide range of machine-learning architectures and training procedures into a shared mathematical language and a small set of reusable implementation patterns. Instead of treating MLPs, CNNs, Transformers, GNNs, mixture-of-experts systems, and multi-task models as unrelated species, the document treats them as different settings of the same typed-state, routed-relation program.

The scope of the current draft is practical rather than encyclopedic. Section 1 formalizes architecture specifications and shows how common model families fit the same state / relation / routing / update template. Section 2 does the same for optimization specifications, including initialization, descent geometry, hyperparameter optimization, and architecture search. Section 3 makes the objective layer explicit by treating targets, losses, metrics, constraints, and model-selection rules as a specification in their own right. The goal is to provide an internal design map that is mathematically coherent, implementation-oriented, and easy to extend as the registry grows.

The distinctive claim of Machinator is not merely that many architectures can be rewritten in a common message-passing idiom. It is that the same typed routed specification can also drive module construction, role-aware initialization, optimizer parameter grouping, conditional hyperparameter spaces, and architecture-search materialization.

The code fragments are implementation sketches rather than full standalone programs. All code shown assumes a Python environment with the following dependencies available: `python>=3.10`, `torch`, and the standard-library modules `math`, `dataclasses`, `typing`, and `collections`. When sampling-based search snippets are used, assume ordinary imports for `torch.nn.functional` as `F` and `Categorical` from `torch.distributions`. The hyperparameter-optimization examples additionally assume an Optuna-style trial interface, with `optuna` or a compatible backend such as `ray[tune]` available when those snippets are executed.

Context and Motivation

Modern deep learning is usually presented as a collection of named families: multilayer perceptrons, convolutional networks, Transformers, graph neural networks, mixture-of-experts systems, set and slot-based models, query-based decoders, and multi-task architectures. Training practice is described with a parallel fragmentation: initialization schemes, optimizer families, gradient modifications, hyperparameter optimization, and neural architecture search are often introduced as

separate engineering topics. That taxonomy is historically useful, but it obscures a simpler underlying structure. Across these families, the recurring design questions are the same: what kinds of states exist, which states may interact, how messages are routed and aggregated, how local updates are applied, how outputs are read out, and how the resulting computation is initialized, optimized, tuned, and optionally searched.[1, 2, 3, 5, 20, 24, 29, 37, 41, 55, 71, 80]

Machinator is organized around that common substrate. On the architecture side, it treats MLPs, CNNs, attention models, graph models, expert-routed modules, adapters, hypernetworks, and multi-task sharing patterns as different instantiations of a typed routed program rather than as unrelated species.[9, 14, 15, 20, 25, 26, 29, 32, 33, 38, 39, 40] On the optimization side, it treats initialization, descent geometry, gradient aggregation, schedule design, hyperparameter optimization, and architecture search as operators attached to that same specification rather than as ad hoc training-script folklore.[42, 43, 44, 52, 56, 61, 78, 79, 81, 82, 83, 84, 88] The goal is not to erase genuine differences in inductive bias, scaling behavior, or deployment cost. It is to express those differences in a form that is explicit, comparable, and directly usable by a configurable implementation.

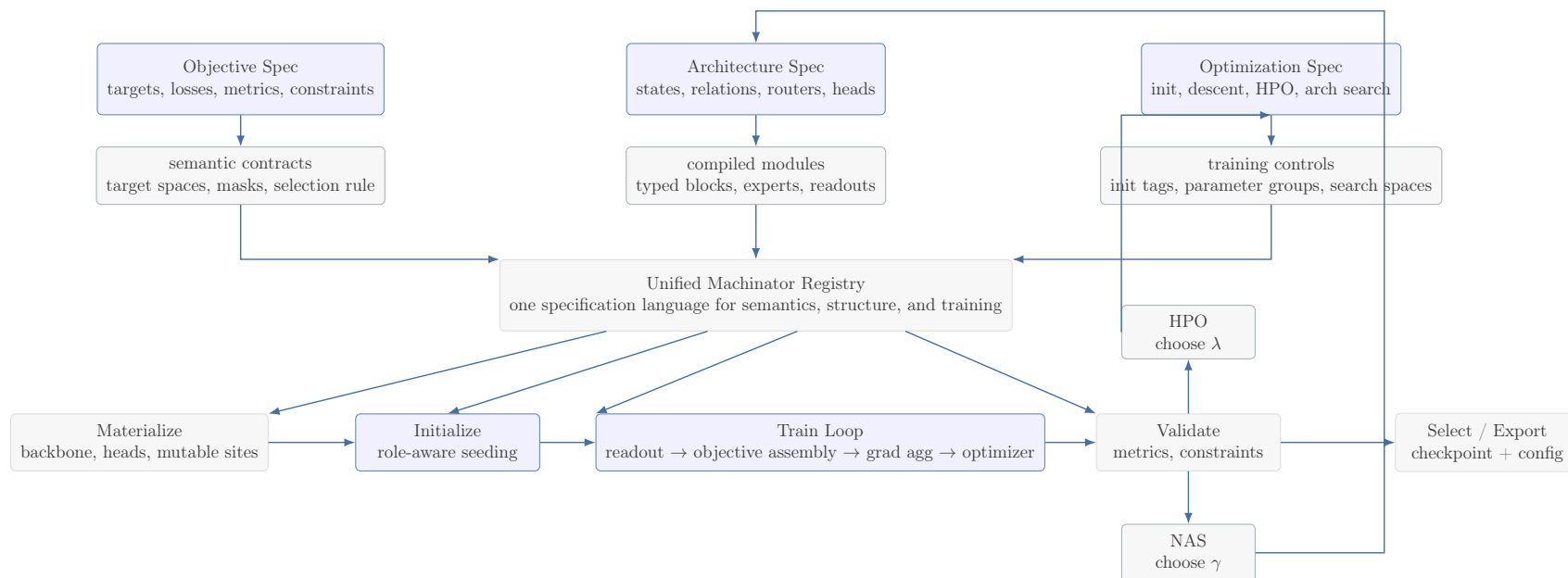
The motivation is therefore twofold. First, a shared formal language provides conceptual compression: it becomes easier to compare model families, explain hybrids, and state exactly where an inductive bias enters, whether through state geometry, relation structure, routing, symmetry constraints, or update rules. Second, the same language provides implementation leverage: once module roles and relations are explicit, the registry can support reusable layers, role-aware initialization, optimizer parameter grouping, conditional hyperparameter spaces, and architecture-search machinery without hard-coding a different framework for each new model family.[6, 7, 8, 30, 31, 36, 49, 57, 75, 76, 77, 85, 86, 87] In that sense, Machinator is meant to serve not only as a descriptive taxonomy, but as a design interface for building, modifying, and searching over a large space of architectures and training systems from one coherent specification.

Contents

Preface	1
Context and Motivation	1
1 Architecture Specifications	5
1.1 Shared Mathematical Vocabulary	5
1.2 Reference Layer Template	7
1.3 State-Slot Construction and Interface Design	8
1.4 Connectivity and Computation Topology	9
1.5 Local and Spatial Operators	10
1.6 Attention and Sequence Relation Operators	11
1.7 Relational, Graph, and Higher-Order Architectures	12
1.8 Routing, Modularity, and Conditional Computation	12

1.9	Symmetry and Geometric Bias	14
1.10	Multi-Task and Multi-Head Sharing	14
1.11	Compact Specification View	14
1.12	Worked Instantiation I: Transformer Encoder	16
1.13	Worked Instantiation II: Multi-Task MMoE / PLE Hybrid	17
2	Optimization Specifications	19
2.1	How the Optimization Families Fit Together	19
2.2	Initialization and Pre-Descent Calibration	21
2.3	Gradient Descent, Optimizer State, and Update Geometry	23
2.4	Hyperparameter Optimization	28
2.5	Architecture Optimization	30
3	Objective Specifications	32
3.1	How Objective Families Fit Together	33
3.2	Target Spaces and Readout Contracts	34
3.3	Loss Assembly, Regularizers, and Auxiliary Terms	35
3.4	Metrics, Constraints, and Model Selection	35
3.5	Compact Objective View	35
	References	37

Visual: Machinator Global Specification Pipeline



1 Architecture Specifications

The design goal of Machinator is to treat named model families as configurations of a single mathematical program rather than as unrelated species. The point of the outline is not to collect fashionable architecture names. The point is to describe a common representation in which MLPs, CNNs, Transformers, GNNs, mixture-of-experts systems, and multi-task models differ only by their choices of state types, relation patterns, routing laws, update operators, and readouts.[1, 2]

In that view, an architecture specification is a compact answer to five questions:

- What kinds of state exist at each depth?
- Which states may communicate with which other states?
- How are messages routed and aggregated?
- How is each local state updated after aggregation?
- How are outputs read from the final state?

The practical payoff is that a broad architecture catalog can be expressed with small amounts of reusable code. One configurable layer, if it is written at the right level of abstraction, becomes enough to instantiate many architectures by changing only wiring, masks, gates, and per-type update blocks.

Comparison Table: Architecture Family Summary

Family	State types	Relation pattern	Routing	Update / topology	Typical heads
MLP	Dense slot set	All-to-all dense	Fixed learned weights	Pointwise nonlinear stack or residual MLP	Classifier, regressor
CNN	Grid cells and channels	Local stencil on a grid	Fixed, translation-tied	Conv blocks, residuals, pyramids	Spatial map or pooled head
Transformer	Tokens, queries, latents	Dense self- or cross-relations	Data-dependent attention	Residual + norm + FFN	Token, sequence, query heads
GNN	Nodes, edges, globals	Sparse graph or typed relation edges	Fixed or edge attention	Message passing + local update	Node, edge, graph heads
MoE	Host states plus expert bank	Slot-to-operator mixture	Sparse top- k or soft gate	Expert mixture inside a host block	Same as host backbone
MTL / hybrid	Shared trunk plus task states	Shared, cross-type, or task-to-task	Task weighting or learned gates	Shared core + multiple heads	One readout per task

1.1 Shared Mathematical Vocabulary

Let $\mathcal{T}$ be a finite set of state types such as token, node, task, memory slot, latent slot, or expert-conditioned slot. For each depth ℓ and type $\tau \in \mathcal{T}$, let:

$I_{\ell,\tau}$ be the index set of slots, $E_{\ell,\tau} \cong \mathbb{R}^{d_{\ell,\tau}}$ be the feature space,

and define the typed hidden state field

$$h_{\ell,\tau} : I_{\ell,\tau} \rightarrow E_{\ell,\tau}.$$

This is the basic normalization step. A layer is not just a vector. It is a collection of typed fields over structured index sets. Once that is admitted, the rest of the architecture catalog becomes an exercise in specifying relations over those fields.

For each relation r , define:

- a source type tuple $S_r = (\tau_1, \dots, \tau_k)$,
- a target type $T_r = \tau$,
- an incidence set

$$\Omega_r \subset I_{\ell,\tau_1} \times \dots \times I_{\ell,\tau_k} \times I_{\ell+1,\tau},$$

- a message map ψ_r ,
- a routing law α_r ,
- an aggregation operator Agg_r ,
- and an action $\odot$ that scales, gates, or otherwise modulates messages.

The routed contribution of relation r to target slot i is

$$m_r(i) = \text{Agg}_{(j_1, \dots, j_k, i) \in \Omega_r} \alpha_r(j_1, \dots, j_k, i; h_\ell) \odot \psi_r(h_{\ell,\tau_1}(j_1), \dots, h_{\ell,\tau_k}(j_k)).$$

If conditional computation is present, expert outputs can be written as

$$e_{\ell,\tau}(i) = \sum_{q \in \mathcal{E}_{\ell,\tau}} \pi_{\ell,\tau,q}(i; h_\ell) f_{\ell,\tau,q}(h_{\ell,\tau}(i)),$$

where π is a gate over expert operators and f_q is the q th expert.

The generic local update then becomes

$$h_{\ell+1,\tau}(i) = U_{\ell,\tau}(h_{\ell,\tau}^{\text{skip}}(i), \{m_r(i) : T_r = \tau\}, e_{\ell,\tau}(i), \xi_{\ell,\tau}(i)),$$

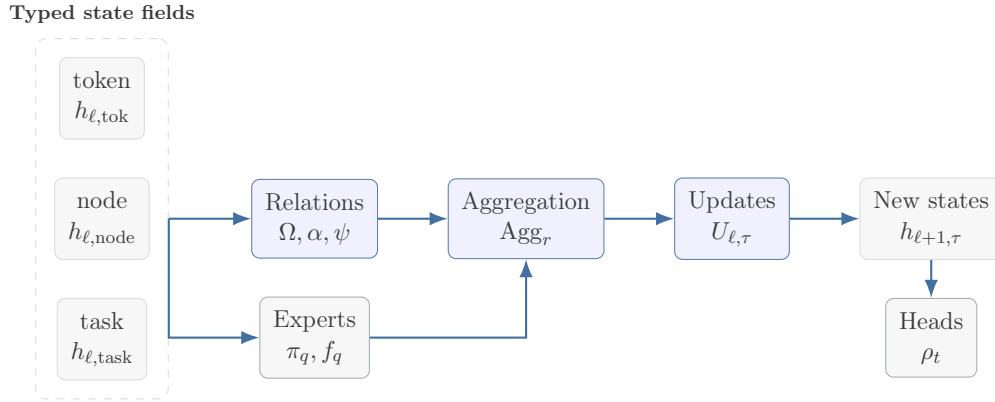
where $\xi_{\ell,\tau}(i)$ may include positional encodings, masks, task identifiers, structural metadata, or side information.

This is the core collating philosophy: architecture families differ by how they instantiate the following tuple, not by whether they are fundamentally different objects:

$$(I, \Omega, \alpha, \psi, \text{Agg}, U, \rho).$$

It is useful to keep the abstraction boundaries explicit. A *relation* specifies admissible source-target tuples together with a message proposal ψ and routing law α . A *topology* specifies how routed contributions are composed across depth, scale, or branch structure once those relations are available. An *update map* U specifies the local post-aggregation state transition at a target slot. Structural priors such as invariance or equivariance restrict the legal form of relations and updates, while readouts ρ act only on final states. Stating these boundaries once prevents attention, convolution, skip topology, and symmetry constraints from being blurred into one vague bucket of “architecture choice.”

Visual: Machinator Core



1.2 Reference Layer Template

The mathematics above is intentionally matched to a compact implementation template. The layer below is small, but it already expresses typed relations, conditional experts, and per-type update maps.

Listing 1: Configurable typed layer

```
class ConfigurableLayer(nn.Module):
    def __init__(self, relations, updates, experts=None):
        super().__init__()
        self.relations = nn.ModuleList(relations)
        self.updates = nn.ModuleDict(updates)
        self.experts = nn.ModuleDict(experts or {})

    def forward(self, states, masks=None):
        contributions = {name: [] for name in states}
        aux = {}

        for rel in self.relations:
            msg, rel_aux = rel(states, masks=masks)
            contributions[rel.dst_type].append(msg)
            aux[rel.name] = rel_aux

        for name, bank in self.experts.items():
            moe_msg, moe_aux = bank(states[name])
            contributions[name].append(moe_msg)
```

```

    aux[f"{name}_moe"] = moe_aux

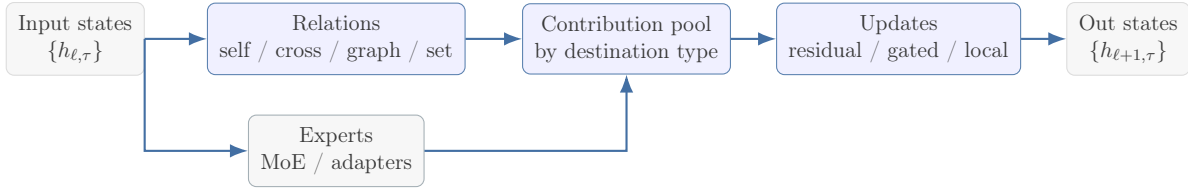
    new_states = {}
    for name, h in states.items():
        new_states[name] = self.updates[name](h, contributions[name])

    return new_states, aux

```

The implementation is compact because the architecture-specific complexity has been moved into relation modules, masks, routers, experts, and readout heads. That is exactly what the formalism is designed to expose.

Visual: Configurable Layer Flow



1.3 State-Slot Construction and Interface Design

The first architecture choice is the state inventory itself. A model may carry token states, node states, set elements, learned latent slots, query slots, object slots, task states, or memory states. Formally, this is the choice of the type set $\mathcal{T}$ and the index families $I_{\ell, \tau}$.

Sequence and dense-vector models. An MLP can be written with a single type and dense incidence:

$$h_{\ell+1}(i) = \sigma \left(b_i^{(\ell)} + \sum_{j \in I_{\ell}} W_{ij}^{(\ell)} h_{\ell}(j) \right).$$

This is the fully connected, fixed-routing case.

Set encoders. For sets, the ordering of elements should not matter. A Deep Sets style encoder is therefore

$$z_j = \phi(x_j), \quad s = \sum_{j \in I_{\text{set}}} z_j, \quad y = \rho(s).$$

The invariant pooling operator is the key architectural constraint.

Latent slots and query slots. Slot Attention, Perceiver, and DETR-like query systems introduce a learned slot or query index set I_{slot} with cross-type relations:

$$h_{\text{slot}}^+(i) = U_{\text{slot}} \left(h_{\text{slot}}(i), \sum_{j \in I_{\text{input}}} \alpha(i, j) W_V h_{\text{input}}(j) \right).$$

This same form covers learned object queries, latent bottlenecks, and output-query decoders. The distinction lies in the semantics of the slot set and the chosen readout.[\[3, 4, 5, 6, 7, 8\]](#)

Listing 2: Cross-type relation for latent or query slots

```
class RelationModule(nn.Module):
    def __init__(self, name, src_type, dst_type, router, messenger, out_proj, mask_key=
        None):
        super().__init__()
        self.name = name
        self.src_type = src_type
        self.dst_type = dst_type
        self.router = router
        self.messenger = messenger
        self.out_proj = out_proj
        self.mask_key = mask_key or name

    def forward(self, states, masks=None):
        src = states[self.src_type]
        dst = states[self.dst_type]
        rel_mask = None if masks is None else masks.get(self.mask_key)
        alpha = self.router(src, dst, mask=rel_mask)
        messages = self.messenger(src, dst)
        if messages.ndim == 3:
            messages = messages.unsqueeze(1).expand(-1, dst.size(1), -1, -1)
        agg = torch.einsum("bij,bijd->bid", alpha, messages)
        return self.out_proj(agg), {"alpha": alpha, "mask_key": self.mask_key}
```

1.4 Connectivity and Computation Topology

Topology describes how blocks are composed once routed contributions are available. In the Machinator view, topology is not a new family of message operators. It is a choice of update map $U_{\ell,\tau}$.

Residual topology. The residual case is

$$h_{\ell+1,\tau}(i) = h_{\ell,\tau}(i) + F_{\ell,\tau}(i).$$

This should be treated as the default stable composition rule. The identity path is explicit, and the learned transformation is a correction term.

Highway topology. Highway-style updates gate the skip path:

$$h_{\ell+1,\tau}(i) = t_{\ell,\tau}(i) \odot F_{\ell,\tau}(i) + (1 - t_{\ell,\tau}(i)) \odot h_{\ell,\tau}(i),$$

where $t_{\ell,\tau}(i) = \sigma(G_{\ell,\tau}(h_{\ell,\tau}(i)))$.

Dense topology. Dense connectivity replaces additive skip composition with feature accumulation:

$$\tilde{h}_{\ell}(i) = [h_0(i) \mid h_1(i) \mid \cdots \mid h_{\ell}(i)], \quad h_{\ell+1}(i) = F_{\ell}(\tilde{h}_{\ell}(i)).$$

This changes memory growth, parameter usage, and the effective information path.

Encoder-decoder and feature pyramids. Multi-scale models add relations across depth and resolution. In a feature pyramid,

$$P_\ell = L_\ell(C_\ell) + \text{Up}(P_{\ell+1}),$$

which is simply a cross-scale relation with a top-down update rule. U-Net style skip fusion is another instance of the same pattern.[\[9, 10, 11, 12, 13\]](#)

Listing 3: Residual, highway, and dense update motifs

```
class ResidualBlock(nn.Module):
    def __init__(self, f):
        super().__init__()
        self.f = f

    def forward(self, x):
        return x + self.f(x)

class HighwayBlock(nn.Module):
    def __init__(self, dim, f):
        super().__init__()
        self.f = f
        self.gate = nn.Linear(dim, dim)

    def forward(self, x):
        t = torch.sigmoid(self.gate(x))
        return t * self.f(x) + (1.0 - t) * x
```

1.5 Local and Spatial Operators

Spatial models are characterized by how Ω_r is constrained by locality and by how parameters are shared across offsets or neighborhoods.

Convolutional locality. On a grid index set, a convolution is

$$h_{\ell+1}(u) = \sigma \left(b + \sum_{\delta \in \Delta} K_\delta h_\ell(u + \delta) \right),$$

where Δ is a local stencil and the same kernel weights are reused across locations. The actual architectural content is local incidence plus translation-tied parameters.

Dilated and causal locality. Dilated convolution changes the neighbor set to

$$\mathcal{N}_d(u) = \{u + d\delta : \delta \in \Delta\},$$

while causal convolution restricts Ω_r so future indices are excluded.

Depthwise-separable and channel-reweighted variants. Depthwise separable convolution factorizes spatial mixing and channel mixing:

$$h'_c(u) = \sum_{\delta \in \Delta} K_{c,\delta} h_c(u + \delta), \quad h_{c'}^+(u) = \sum_c P_{c'c} h'_c(u).$$

Channel recalibration modules then add a learned multiplicative gate

$$g = \sigma(W_2 \phi(W_1 \text{GAP}(h))), \quad h^+ = g \odot h.$$

These are best understood as local relations plus local update modifiers, not as disconnected architectural inventions.[14, 15, 16, 17, 18, 19]

1.6 Attention and Sequence Relation Operators

Attention is the dense, data-routed relation case. With source states s_j and destination states d_i ,

$$q_i = W_Q d_i, \quad k_j = W_K s_j, \quad v_j = W_V s_j.$$

The routing law is

$$\alpha(i, j) = \text{softmax}_j \left(\frac{q_i^\top k_j}{\sqrt{d_h}} + b_{ij} \right),$$

and the aggregated contribution is

$$m(i) = \sum_j \alpha(i, j) v_j.$$

If source and destination types are the same, this is self-attention. If they differ, it is cross-attention. Relative position terms, rotary structure, or linear attention variants modify either the score bias b_{ij} , the representation of (q_i, k_j) , or the aggregation algorithm used to compute the same basic object.[20, 21, 22, 23]

Listing 4: Dense routed relation in attention form

```
class DotProductRouter(nn.Module):
    def __init__(self, d_src, d_dst, d_model):
        super().__init__()
        self.to_q = nn.Linear(d_dst, d_model)
        self.to_k = nn.Linear(d_src, d_model)

    def forward(self, src, dst, mask=None):
        q = self.to_q(dst)
        k = self.to_k(src)
        scores = q @ k.transpose(-2, -1) / math.sqrt(q.size(-1))
        if mask is not None:
            scores = scores.masked_fill(~mask, float("-inf"))
        return scores.softmax(dim=-1)
```

The transformer block is then just a complete-graph relation over tokens followed by a local update:

$$h'(i) = h(i) + m(i), \quad h^+(i) = h'(i) + \text{FFN}(\text{LN}(h'(i))).$$

1.7 Relational, Graph, and Higher-Order Architectures

Graph models are the sparse relation case. Let $G = (V, E)$ be a graph. Message passing becomes

$$m(i) = \text{Agg}_{j:(j,i) \in E} \psi(h(i), h(j), e_{ji}), \quad h^+(i) = \phi(h(i), m(i)).$$

Graph convolution. GCN-style propagation can be written as

$$H^+ = \sigma(\hat{A}HW),$$

where $\hat{A}$ is a normalized adjacency with self-loops.

Graph attention. GAT-style propagation simply replaces fixed normalization with learned routing:

$$\alpha(i, j) = \text{softmax}_{j \in \mathcal{N}(i)} a(Wh(i), Wh(j)), \quad m(i) = \sum_{j \in \mathcal{N}(i)} \alpha(i, j) Wh(j).$$

Hypergraph and typed-relation models. If relations involve tuples beyond pairs, the incidence set Ω_r is a hyperedge relation rather than an edge set. Heterogeneous graph models then arise by making type-dependent choices of message map, routing law, and parameter bank.[\[24, 25, 26, 2, 27, 28\]](#)

Listing 5: Masked message passing over a sparse relation

```
class UniformMaskedRouter(nn.Module):
    def forward(self, src, dst, mask):
        weights = mask.float()
        denom = weights.sum(dim=-1, keepdim=True).clamp_min(1.0)
        return weights / denom

class SourceLinearMessage(nn.Module):
    def __init__(self, d_in, d_out):
        super().__init__()
        self.proj = nn.Linear(d_in, d_out)

    def forward(self, src, dst):
        msg = self.proj(src)
        return msg.unsqueeze(1).expand(-1, dst.size(1), -1, -1)
```

In other words, a transformer is message passing on a complete graph, and a GNN is message passing on a constrained graph. The same layer skeleton supports both once routing and incidence are made explicit.

1.8 Routing, Modularity, and Conditional Computation

Mixture-of-experts systems add routing over operator banks instead of, or in addition to, routing over source positions. If f_q denotes expert q , then

$$e(i) = \sum_{q=1}^Q \pi_q(i; h) f_q(h(i)),$$

with either dense softmax gates or sparse top- k routing.

This is best read as attention over functions. The source of variability is not which token is consulted, but which transformation is activated for the current slot.

Adapters and hypernetworks. Adapters are small expert-like local operators inserted into a shared backbone:

$$h^+ = h + W_{\text{up}} \phi(W_{\text{down}} h).$$

Hypernetworks move conditionality into parameter generation:

$$\theta(i) = g(c(i)), \quad h^+(i) = f_{\theta(i)}(h(i)).$$

Both fit naturally inside the same conditional-update abstraction. [\[29, 30, 31, 32, 33\]](#)

Listing 6: Slot-wise sparse expert bank with top- k dispatch

```
class SparseExpertBank(nn.Module):
    def __init__(self, experts, gate, k=2):
        super().__init__()
        self.experts = nn.ModuleList(experts)
        self.gate = gate
        self.k = k

    def forward(self, x):
        # x: [batch, slots, dim]
        b, n, d = x.shape
        flat_x = x.reshape(b * n, d)

        logits = self.gate(flat_x)
        topk_logits, topk_idx = logits.topk(self.k, dim=-1)
        topk_probs = topk_logits.softmax(dim=-1)

        flat_out = torch.zeros_like(flat_x)
        expert_load = torch.zeros(len(self.experts), device=x.device)

        for expert_id, expert in enumerate(self.experts):
            token_idx, choice_idx = (topk_idx == expert_id).nonzero(as_tuple=True)
            if token_idx.numel() == 0:
                continue

            expert_in = flat_x.index_select(0, token_idx)
            expert_out = expert(expert_in)
            weights = topk_probs[token_idx, choice_idx].unsqueeze(-1)
            flat_out.index_add_(0, token_idx, weights * expert_out)
            expert_load[expert_id] = token_idx.numel()

        y = flat_out.view(b, n, d)
        aux = {
            "router_logits": logits.view(b, n, -1),
            "topk_idx": topk_idx.view(b, n, self.k),
            "topk_probs": topk_probs.view(b, n, self.k),
            "expert_load": expert_load,
        }
        return y, aux
```

1.9 Symmetry and Geometric Bias

Symmetry priors do not introduce a separate computational species. They impose structure on either the parameter sharing rule or the admissible update map. The general condition is equivariance:

$$f(g \cdot x) = g \cdot f(x) \quad \text{for every } g \in G.$$

For convolution, the group G is translation. For permutation-invariant set models, the relevant action is permutation of set elements. For equivariant graph or geometric models, the group may include translations, rotations, reflections, or permutations of nodes.

An $E(n)$ -equivariant update, for example, separates feature messages from coordinate updates:

$$m_{ij} = \psi_h(h_i, h_j, \|x_i - x_j\|^2), \quad x_i^+ = x_i + \sum_{j \neq i} (x_i - x_j) \psi_x(m_{ij}).$$

The important point for the outline is that symmetry is a constraint layered on top of the same routed relational program. It changes the legal form of ψ , α , or parameter sharing, not the overall abstraction.[\[34, 35, 36\]](#)

1.10 Multi-Task and Multi-Head Sharing

Multi-task learning is primarily a readout and sharing problem. In the simplest hard-sharing case,

$$s = F(x), \quad y_t = \rho_t(s), \quad t \in \mathcal{T}_{\text{task}}.$$

The shared trunk is architecture-agnostic. It may itself be an MLP, Transformer, GNN, or hybrid typed-state model.

More structured multi-task systems add task-indexed state and task-to-task relations:

$$\tilde{h}_t^\ell = \sum_{u \in \mathcal{T}_{\text{task}}} C_{tu}^\ell h_u^\ell, \quad h_t^{\ell+1} = f_t^\ell(\tilde{h}_t^\ell).$$

If the mixing coefficients become data dependent,

$$\tilde{h}_t^\ell = \sum_{u \in \mathcal{T}_{\text{task}}} \alpha_{tu}^\ell(h^\ell) W_{tu} h_u^\ell,$$

then cross-stitch style transfer becomes a routed relation among task states.

In Machinator terms, multi-task learning is not a separate branch of the architecture tree. It is a choice to add multiple readout heads, task-specific loss terms, and optionally explicit task state types.[\[37, 38, 39, 40\]](#)

1.11 Compact Specification View

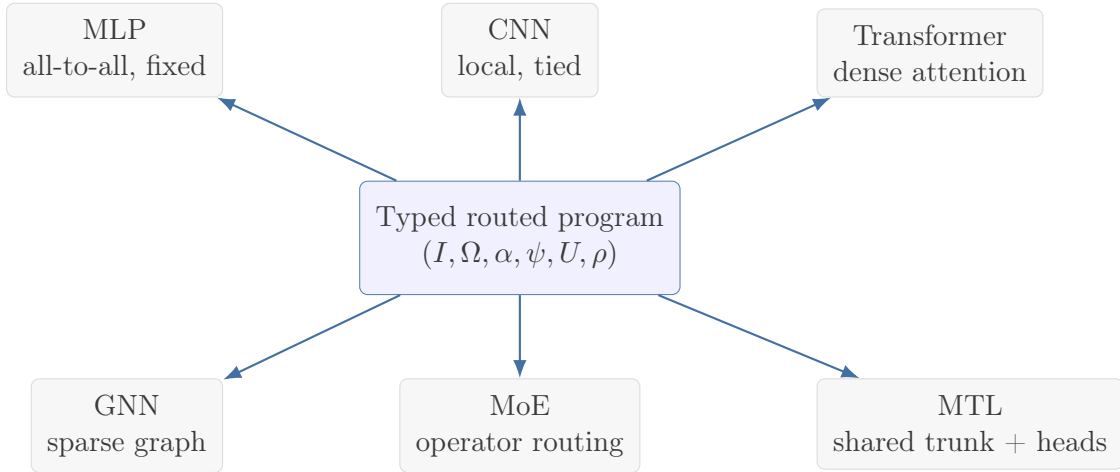
The full architecture specification can therefore be summarized as a typed routed program:

$$\mathfrak{N} = \left(\mathcal{T}, \{I_{\ell, \tau}\}, \{E_{\ell, \tau}\}, \{R_\ell\}, \{U_{\ell, \tau}\}, \{\rho_t\} \right).$$

Comparison Table: Architecture-to-Formalism Mapping

Family	State geometry I	Connectivity Ω	Routing α	Message ψ	Update U
MLP	Dense layer indices	Complete bipartite across layers	Fixed learned weights	Linear map $W_{ij}h_j$	Nonlinearity, bias, residual
CNN	Grid locations and channels	Local offset stencil	Fixed and translation tied	Kernel-weighted local mix	Conv block, pooling, residual
Transformer	Token, query, or latent indices	Complete graph or cross-type pairs	Attention scores from $QK^\top$	Value vectors or projected source states	Residual, norm, FFN
GNN	Node, edge, or global sets	Sparse graph or typed relation edges	Fixed normalize or edge attention	Edge-conditioned message	Message-passing update
MoE	Host slots plus expert index	Slot-to-expert incidence	Gate or top- k router	Expert output $f_q(h_i)$	Residual mixture update
MTL	Shared slots plus task index	Shared trunk plus task links	Task weights or task gates	Task-conditioned transfer message	Shared update + task heads

Visual: One Formalism, Many Architecture Families



Familiar models arise by configuration:

- An MLP uses one type, dense fixed routing, and pointwise nonlinear updates.
- A CNN uses grid-structured indices, local incidence, and translation-tied parameters.
- A Transformer uses dense token-to-token relations with data-dependent routing.
- A GNN uses sparse graph-restricted relations over node states.
- A mixture-of-experts layer routes each slot through a bank of local operators.
- A multi-task model adds multiple heads and optionally task-indexed relations.

That is the core philosophy of the architecture section: the catalog is broad at the naming level but narrow at the implementation level. Once the right mathematical object is chosen, the code can stay small while the expressible design space stays large.

The same abstraction is also the implementation handle for the optimization layer. Once states, relations, routers, experts, and heads are explicit, the registry can attach role-aware initialization policies, optimizer parameter groups, conditional hyperparameter spaces, and mutable search sites to the same specification rather than scattering those decisions across unrelated training scripts. The next section makes that second claim formal.

1.12 Worked Instantiation I: Transformer Encoder

To make the abstraction concrete, consider a standard pre-norm Transformer encoder.[\[20\]](#) The type set is

$$\mathcal{T} = \{\text{tok}\}, \quad I_{\ell, \text{tok}} = \{1, \dots, n\},$$

with one token state field per depth. The relation set at each layer contains a dense self-attention relation and a pointwise feedforward update. The attention relation uses complete token-to-token incidence

$$\Omega_{\ell, \text{attn}} = I_{\ell, \text{tok}} \times I_{\ell, \text{tok}},$$

dot-product routing, value messages, and sum aggregation. The local update is pre-norm residual composition:

$$h'_\ell(i) = h_\ell(i) + m_{\ell, \text{attn}}(i), \quad h_{\ell+1}(i) = h'_\ell(i) + \text{FFN}(\text{LN}(h'_\ell(i))).$$

Readout may be tokenwise, pooled, or attached to a dedicated classification token. The important point is that nothing in this description requires a separate architecture language beyond typed states, dense relations, routed aggregation, and local updates.

Using the typed schema formalized in Section 2, the same encoder can be written as:

Listing 7: End-to-end Machinator view of a Transformer encoder

```
transformer_encoder = MachinatorSpec(
  architecture={
    "types": ["tok"],
    "relations": [
      {
        "name": "self_attn",
        "src": "tok",
        "dst": "tok",
        "pattern": "complete",
        "router": "dot_product",
        "message": "value_proj",
        "update": "pre_norm_residual_attn",
      },
      {
        "name": "ffn",
        "src": "tok",
        "dst": "tok",
        "pattern": "pointwise",
        "router": "identity",
        "message": "two_layer_mlp",
        "update": "pre_norm_residual_ffn",
      },
    ],
  },
)
```



```

    "heads": {"kind": "token_or_cls"},
  },
  optimization=OptimizationSpec(
    objective=ObjectiveSpec(
      task_weights={"main": 1.0},
      regularizers={"weight_decay": 1.0},
      constraints={},
    ),
    init=InitSpec(
      policy="role_aware",
      overrides={
        "q_proj": {"scheme": "deepnorm"},
        "k_proj": {"scheme": "deepnorm"},
        "v_proj": {"scheme": "xavier_uniform"},
        "out_proj": {"scheme": "deepnorm"},
      },
    ),
    descent=DescentSpec(
      base_optimizer="adamw",
      preconditioner="diag",
      schedule="warmup+cosine",
      grad_transforms=["clip_norm"],
      averaging="ema",
    ),
  ),
)

```

In this instantiation, initialization tags, optimizer groups, and search knobs all attach to the same objects already introduced by the architecture spec. The encoder is therefore a complete example of the paper’s main claim: the same specification language describes both the model and the handles used to train it.

1.13 Worked Instantiation II: Multi-Task MMoE / PLE Hybrid

The more interesting stress test is a shared-trunk multi-task model with expert routing.[\[39, 40, 29\]](#) Let the type set include shared feature states and task-indexed states:

$$\mathcal{T} = \{\text{shared}, \text{task}_1, \dots, \text{task}_K\}.$$

Each layer may contain a shared relation block, task-conditioned expert routing, and optional task-to-task transfer. A compact form is

$$\begin{aligned}
 s^{\ell+1} &= U_{\text{shared}}^\ell(s^\ell, m_{\text{shared}}^\ell, e_{\text{shared}}^\ell), \\
 t_k^{\ell+1} &= U_k^\ell(t_k^\ell, m_k^\ell, \sum_q \pi_{k,q}^\ell(x) f_q^\ell(s^\ell), \sum_{u \neq k} \alpha_{ku}^\ell(x) W_{ku} t_u^\ell).
 \end{aligned}$$

The losses are task-indexed,

$$\mathcal{J} = \sum_{k=1}^K \lambda_k \mathcal{L}_k + \lambda_{\text{route}} L_{\text{balance}} + \lambda_z L_z,$$

so the optimizer also needs an explicit task-gradient aggregation policy and router-specific auxiliaries.

Using the same specification schema, the hybrid can be written as:

Listing 8: End-to-end Machinator view of an MMoE / PLE-style hybrid

```
mmoe_hybrid = MachinatorSpec(
  architecture={
    "types": ["shared", "task_a", "task_b"],
    "relations": [
      {"name": "shared_trunk", "src": "shared", "dst": "shared", "pattern": "dense"},
      {"name": "shared_to_task", "src": "shared", "dst": "task_a", "pattern": "cross_type"},
      {"name": "shared_to_task", "src": "shared", "dst": "task_b", "pattern": "cross_type"},
      {"name": "task_transfer_a2b", "src": "task_a", "dst": "task_b", "pattern": "gated"},
      {"name": "task_transfer_b2a", "src": "task_b", "dst": "task_a", "pattern": "gated"},
    ],
    "routing": {"kind": "sparse_experts", "k": 2},
    "heads": {"kind": "mmoe"},
  },
  optimization=OptimizationSpec(
    objective=ObjectiveSpec(
      task_weights={"task_a": 1.0, "task_b": 1.0},
      regularizers={"router_aux_weight": 1e-2, "z_loss": 1e-3},
      constraints={},
    ),
    init=InitSpec(
      policy="role_aware",
      overrides={
        "router": {"scheme": "small_logits"},
        "expert": {"scheme": "xavier_uniform"},
        "head": {"scheme": "xavier_uniform"},
      },
    ),
    descent=DescentSpec(
      base_optimizer="adamw",
      preconditioner="diag",
      schedule="warmup+cosine",
      loss_aggregator="pcgrad",
      grad_transforms=["clip_norm"],
      router_aux="switch_load_balance",
    ),
    hpo=HPOSpec(
      backend="optuna",
      budget_key="steps",
      search_space={
        "gate_dropout": [0.0, 0.4],
        "router_aux_weight": [1e-4, 1e-1],
        "num_shared_experts": [4, 16],
      },
    ),
  ),
)
```

),
),
)

This hybrid case is useful because it forces the framework to unify typed states, routed operator selection, multiple heads, multiple losses, router-specific regularization, and conditional search spaces in one object. If the same specification cannot carry this example cleanly, then the claimed unification is cosmetic rather than structural.

2 Optimization Specifications

Architecture determines what is computed. Optimization determines how that architecture is initialized, how its parameters move, which training knobs are searched, and when the architecture itself is allowed to change. In Machinator, optimization is therefore not a miscellaneous appendix. It is the operational second specification layer attached to the same typed relational program introduced in Section 1.[\[41, 55, 71, 80\]](#)

2.1 How the Optimization Families Fit Together

Let A denote an architecture specification, let θ denote ordinary trainable weights, let γ denote optional architecture-search variables, and let λ denote optimizer or training hyperparameters. The total training objective at step t can be written as

$$\mathcal{J}_t(\theta, \gamma; \lambda) = \sum_m w_m(\lambda) \mathcal{L}_t^{(m)}(\theta, \gamma) + \lambda_{\text{reg}} R(\theta, \gamma) + \lambda_{\text{cost}} C(\gamma).$$

The full optimization stack then operates at four connected timescales:

$$\begin{aligned} \theta_0 &= \mathcal{I}(A, \text{tags}, \mathcal{B}_0, \theta^{\text{pre}}), \\ (\theta_{t+1}, s_{t+1}) &= \mathcal{D}_t(\theta_t, s_t, \nabla_{\theta} \mathcal{J}_t; \lambda), \\ \lambda^* &= \arg \min_{\lambda \in \Lambda} F(\lambda) \quad \text{subject to } c_j(\lambda) \leq 0, \\ \gamma^* &= \arg \min_{\gamma \in \mathcal{A}} G(\gamma) \quad \text{or} \quad (\theta^*, \gamma^*) = \arg \min_{\theta, \gamma} \mathcal{J}_{\text{val}}(\theta, \gamma). \end{aligned}$$

$$\mathfrak{D} = (\mathcal{J}, \mathcal{I}, \mathcal{G}, \Gamma, \Psi, P, S, \mathcal{H}, \mathcal{Q}),$$

where $\mathcal{J}$ is objective assembly, $\mathcal{I}$ is initialization, $\mathcal{G}$ is loss or task-gradient aggregation, Γ is the gradient-transform chain, Ψ is optimizer-state evolution, P is the preconditioner geometry, S is the schedule and iterate-averaging policy, $\mathcal{H}$ is the hyperparameter-search operator, and $\mathcal{Q}$ is the architecture-search operator. Optimization families differ by how they instantiate this tuple, not by belonging to unrelated toolboxes.

The first line is initialization. The second is ordinary descent. The third is hyperparameter optimization over whole training runs. The fourth is architecture optimization over structural variables such as operator choice, depth, width, routing, or sparsity. These are not independent islands.

Initialization depends on module role, descent depends on how losses and gradients are structured by the architecture, HPO depends on the conditionals induced by the architecture registry, and architecture optimization is usually a bilevel or black-box outer loop wrapped around the inner descent dynamics.[\[52, 71, 80\]](#)

The clean implementation pattern is to keep these choices as an explicit optimization spec rather than scattering them across constructors and training scripts.

Listing 9: Typed schema for the architecture and optimization specifications

```
@dataclass
class ObjectiveSpec:
    task_weights: dict[str, float]
    regularizers: dict[str, float]
    constraints: dict[str, float]

@dataclass
class InitSpec:
    policy: str
    overrides: dict[str, dict]
    calibration: dict | None = None

@dataclass
class DescentSpec:
    base_optimizer: str
    preconditioner: str = "diag"
    trust_ratio: str = "none"
    schedule: str = "warmup+cosine"
    grad_transforms: list[str] = field(default_factory=list)
    loss_aggregator: str = "sum"
    router_aux: str = "none"
    averaging: str = "none"

@dataclass
class HPOSpec:
    backend: str
    budget_key: str
    search_space: dict

@dataclass
class ArchSearchSpec:
    mode: str
    mutable_sites: list[str] = field(default_factory=list)
    cost_objective: str | None = None

@dataclass
class OptimizationSpec:
    objective: ObjectiveSpec
    init: InitSpec
    descent: DescentSpec
    hpo: HPOSpec | None = None
    arch_search: ArchSearchSpec | None = None

@dataclass
class MachinatorSpec:
```

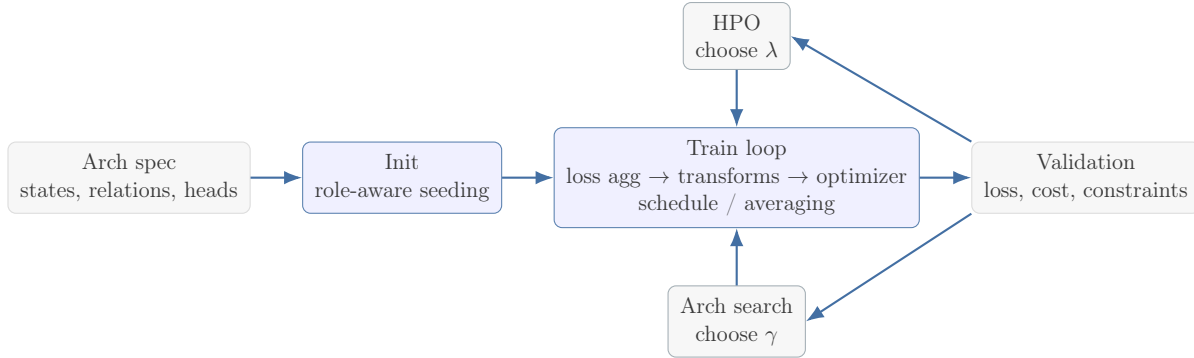
```

architecture: dict
optimization: OptimizationSpec

```

That object is the bridge between Section 1 and Section 2. The architecture spec tells the system what kinds of modules and relations exist. The optimization spec tells the system how those modules are seeded, updated, tuned, and optionally searched. In that sense the optimization layer is not a separate engineering appendix. It is the operational half of the same specification language.

Visual: Optimization Stack and Timescales



2.2 Initialization and Pre-Descent Calibration

Initialization is best understood as a policy over parameter roles, not a blind fill over tensor shapes. A matrix with shape $(4096, 4096)$ might be a query projection, a residual-branch tail, a router, an expert MLP, or a graph message map. Those roles have different aggregation semantics, and therefore different failure modes before the first optimizer step.[41, 42, 43, 44, 49, 46]

Formally, let $z(p)$ be a role tag for parameter tensor p containing fields such as role, relation type, aggregator type, branch identity, section, and depth. Then initialization is a map

$$\theta_0(p) = \mathcal{I}(z(p), A, \mathcal{B}_0, \theta^{\text{pre}}),$$

where $\mathcal{B}_0$ is an optional calibration batch and θ^{pre} is optional pretrained state.

Additive aggregation. If a block is formed by residual or multi-branch addition,

$$y = x + \sum_{b=1}^B F_b(x),$$

then variance accumulates along the summed branches. This is why depth-aware methods bias the network toward the identity map at initialization. Fixup rescales residual branches and zeros the last branch layer; SkipInit and ReZero explicitly write

$$y = x + a_b F_b(x), \quad a_b(0) \approx 0;$$

DeepNorm and T-Fixup rescale specific projections so very deep transformers do not amplify residual paths into instability.[42, 50, 43, 51]

Concatenative aggregation. If a block merges by concatenation,

$$y = [F_1(x) \mid F_2(x) \mid \cdots \mid F_B(x)],$$

the merge itself is not the dangerous point. The important quantity is the fan-in of the *next* mixer, which becomes $\sum_b d_b$. In practice this means ordinary Xavier, Kaiming, or orthogonal fills are often enough as long as the post-concatenation projection is initialized with the true expanded width.

Attention-weighted aggregation. Attention splits initialization roles in a way dense MLP thinking does not capture:

$$q_i = W_Q h_i, \quad k_j = W_K h_j, \quad v_j = W_V h_j.$$

The Q/K pair determines score geometry, while V/O determines output magnitude. That is why transformer-specific schemes target those roles separately: Pre-LN changes the effective initialization landscape, T-Fixup rescales selected attention and feedforward matrices, DeepNorm uses role-specific gains, and sigma-reparameterization stabilizes score spectra directly.

Variable-cardinality reductions. Pooling, set aggregation, graph readout, and degree-varying message passing are often better served by one real calibration batch than by another closed-form variance slogan. If

$$s = \sum_{i=1}^n z_i,$$

then the output scale depends directly on cardinality n . LSUV, data-dependent initialization, AutoInit, GradInit, and MetaInit matter here because they adjust to the actual realized activation statistics of the composed model.[\[44\]](#)

Specialized cases. Sparse MoE routers should be initialized more conservatively than expert MLPs because bad routing logits break training earlier than bad expert activations. Low-rank adapters are also role-specific: LoRA commonly zeros one factor so the base model is initially preserved, while PiSSA initializes the low-rank path from dominant singular directions of a pretrained matrix. Recurrent cores, graph operators, and state-space layers each have their own structure-preserving defaults for the same reason.[\[31, 49, 47, 48, 46\]](#)

Comparison Table: Initialization Policy Families

Policy	Stage	Uses batch?	Role-aware?	Best for	Key idea
Xavier / Kaiming / orthogonal	Tensor fill	No	Usually no	Plain MLP, CNN, generic dense blocks	Preserve variance or singular structure by shape
LSUV / data dependent	Calibration pass	Yes	Indirectly	Variable-cardinality or heterogeneous networks	Rescale activations from one real batch
Fixup / SkipInit / ReZero	Topology aware	No	Yes	Deep residual stacks without heavy normalization	Bias the network toward identity at step zero
DeepNorm / T-Fixup	Role + topology aware	No	Yes	Deep transformers	Rescale residual and attention / FFN projections by role
LoRA / PiSSA style	Adapter or pretrained stage	Sometimes	Yes	Pretrained adaptation	Preserve or align low-rank path to the base model
μ P / unit scaling	Reparam. stage	No	System wide	Width transfer or low precision scaling	Keep scales stable as model size changes

The implementation consequence is that initialization needs access to tagged module roles and to model-level context such as depth, branch count, or pretrained weights.

Listing 10: Role-aware initialization tags

```
@dataclass
class InitTag:
    role: str # q_proj, out_proj, router, expert, head, ...
    relation: str | None = None
    aggregator: str | None = None
    section: str | None = None
    branch_id: str | None = None
    branch_pos: int = 0
    depth: int | None = None

def tag(module: nn.Module, **kwargs) -> nn.Module:
    module.init_tag = InitTag(**kwargs)
    return module
```

This is what allows one framework to support plain tensor fills such as Xavier or Kaiming, topology-aware schemes such as Fixup or DeepNorm, calibration passes such as LSUV, and pretrained-adapter policies such as PiSSA without hard-coding special cases into every block constructor.

2.3 Gradient Descent, Optimizer State, and Update Geometry

Once initialization has placed the model in a sane regime, the next problem is update geometry. For Machinator, the descent specification is not just a choice of optimizer name. It includes the base

update rule, the preconditioner geometry, optional trust-ratio scaling, gradient transforms, cross-loss aggregation, router-specific auxiliaries, the learning-rate schedule, and any iterate-averaging wrapper.[\[52\]](#)

Listing 11: Optimization feature map for the descent layer

```
OPT_SPEC = {
  "base_optimizer": "sgd | adamw | adafactor | muon | lion | lamb | shampoo | ...",
  "preconditioner": "none | diag | factored | kronecker | hessian_diag",
  "trust_ratio": "none | lars | lamb",
  "schedule": "none | warmup+cosine | sgdr | onecycle | schedule_free",
  "grad_transforms": ["clip_norm", "agc", "gradient_centralization", "sam", "lookahead"],
  "loss_aggregator": "sum | uncertainty | gradnorm | mgda | pcgrad | cagrad",
  "router_aux": "none | switch_load_balance | router_z_loss",
  "averaging": "none | ema | swa",
}
```

The generic descent step can be written as

$$\begin{aligned} g_t^{(k)} &= \nabla_{\theta} \mathcal{L}_t^{(k)}(\theta_t, \gamma), & g_t &= \mathcal{G}(g_t^{(1)}, \dots, g_t^{(K)}), \\ \tilde{g}_t &= \Gamma_t(g_t, \theta_t), & s_{t+1} &= \Psi(s_t, \tilde{g}_t, \theta_t; \lambda), \\ u_t &= P_t^{-1}(\tilde{g}_t, s_{t+1}), & \theta_{t+1} &= \theta_t - \eta_t r_t \odot u_t. \end{aligned}$$

Here $\mathcal{G}$ is the loss or task-gradient aggregator, Γ_t is the gradient-transform chain, P_t^{-1} is the preconditioner, and r_t is an optional trust-ratio term.

Loss assembly and multi-loss aggregation. The optimizer does not minimize a bare task loss. It minimizes the assembled objective produced by the architecture and routing choices:

$$\mathcal{J}_t = \sum_k \lambda_k \mathcal{L}_k^{\text{task}} + \lambda_{\text{route}} R_{\text{balance}} + \lambda_z R_z + \lambda_{\text{sym}} R_{\text{equiv}} + \lambda_{\text{sparse}} R_{\text{mask}}.$$

For shared-trunk or multi-head systems, the aggregation map $\mathcal{G}$ itself is an optimization design choice. The plain weighted sum is

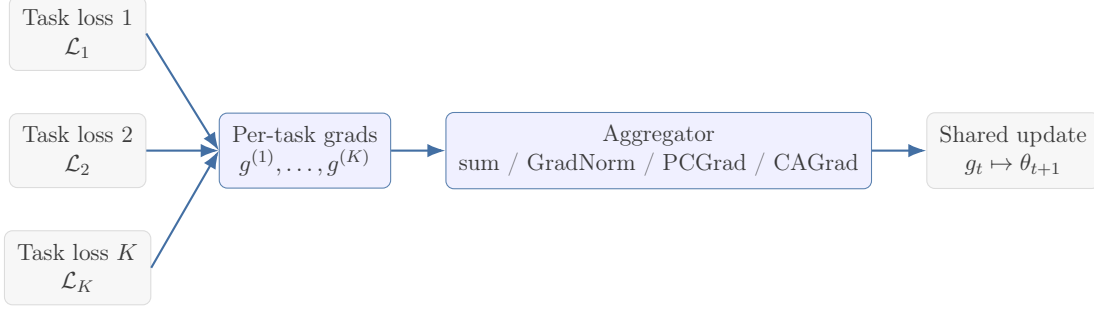
$$g_t = \sum_{k=1}^K w_k g_t^{(k)},$$

while conflict-aware methods modify the task gradients before summation. A PCGrad-style projection, for example, suppresses the component of g_i that directly conflicts with g_j :

$$g_i \leftarrow g_i - \frac{\min(0, \langle g_i, g_j \rangle)}{\|g_j\|^2} g_j.$$

This is the right level of abstraction for MMoE, PLE, cross-stitch, or any learned-branching model with a shared backbone.[\[69, 70\]](#)

Visual: Shared-Trunk Gradient Aggregation



Base update rules and preconditioner geometry. Classical SGD with momentum remains the simplest transport rule:

$$m_t = \beta m_{t-1} + (1 - \beta) \tilde{g}_t, \quad \theta_{t+1} = \theta_t - \eta_t m_t.$$

Adaptive methods introduce a geometry through P_t . In AdamW,

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \tilde{g}_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) \tilde{g}_t^{\odot 2},$$

$$P_t = \text{diag}(\sqrt{\hat{v}_t} + \varepsilon), \quad \theta_{t+1} = (1 - \eta_t \lambda_{\text{wd}}) \theta_t - \eta_t P_t^{-1} \hat{m}_t.$$

This is the diagonal-preconditioned case. Adafactor is the factored-memory variant, where the second moment for a matrix parameter is approximated by lower-order row and column statistics rather than a full dense diagonal buffer. Kronecker or tensor-structured preconditioners such as K-FAC, Shampoo, or SOAP replace the diagonal geometry with richer matrix structure. Muon is another role-specific case: it is best used for 2D hidden matrices, while embeddings, biases, and norms should stay on a conventional optimizer such as AdamW. The correct implementation is therefore often a hybrid, not a single monolithic optimizer.[\[53, 54, 55, 56, 57, 60, 61, 62\]](#)

Comparison Table: Optimizer and Preconditioner Families

Method	Base rule	Geometry	Extra state	Best use case	Notes
SGD + momentum	First-order transport	None	Momentum vector	CNNs, large-batch convnets, simple baselines	Minimal optimizer bias, easy to schedule
AdamW	First-order adaptive	Diagonal second moment	m_t, v_t	Transformers, graph transformers, generic dense stacks	Decoupled decay, durable default
Adafactor	Adam-like adaptive	Factored second moment	Row / column factors	Memory-constrained large models	Lower optimizer-state cost
LARS / LAMB	SGD- or Adam-like	Layerwise trust ratio	Base state + trust scale	Very large batches	Stabilize step size by parameter norm
Muon + AdamW	Hybrid first-order	Role-specific matrix geometry	Split optimizer state	Matrix-heavy hidden layers	Use Muon on 2D hidden matrices only
K-FAC / Shampoo / SOAP	Curvature aware	Kronecker or tensor structured	Larger matrix state	Compute-rich benchmark runs	Richer geometry, more machinery
Sophia / AdaHessian	First-order + curvature estimate	Diagonal Hessian-ish	Adaptive moments + Hessian estimate	Faster large-model experimentation	More curvature than Adam without full Shampoo

Trust-ratio scaling for huge batches. Large-batch training often benefits from an explicit trust ratio at the layer or parameter-group level:

$$r_t^{(\ell)} = \frac{\|\theta_t^{(\ell)}\|}{\|u_t^{(\ell)}\| + \varepsilon}, \quad \theta_{t+1}^{(\ell)} = \theta_t^{(\ell)} - \eta_t r_t^{(\ell)} u_t^{(\ell)}.$$

LARS applies this idea on top of SGD-like updates for convolutional settings. LAMB applies the same principle on top of Adam-like moments and is the more natural large-batch choice for transformer-heavy models.[58, 59]

Gradient transforms and wrappers. Many important “optimizers” are really transforms on top of a base step. Global norm clipping writes

$$\tilde{g}_t \leftarrow g_t \cdot \min\left(1, \frac{c}{\|g_t\|}\right),$$

while adaptive gradient clipping applies the same idea relative to parameter norms:

$$\tilde{g}_t^{(\ell)} \leftarrow g_t^{(\ell)} \cdot \min\left(1, \frac{\lambda \|\theta_t^{(\ell)}\|}{\|g_t^{(\ell)}\| + \varepsilon}\right).$$

Gradient centralization subtracts gradient means before the optimizer state is updated. SAM perturbs the parameters toward a locally sharp direction and differentiates the perturbed loss:

$$\epsilon_t = \rho \frac{g_t}{\|g_t\|}, \quad g_t^{\text{SAM}} = \nabla_{\theta} \mathcal{J}_t(\theta_t + \epsilon_t).$$

Lookahead is another wrapper rather than a base optimizer:

$$\phi \leftarrow \phi + \beta_{\text{LA}}(\theta - \phi), \quad \theta \leftarrow \phi,$$

where ϕ denotes slow weights updated every few fast steps.[64, 65, 66, 67, 68]

Comparison Table: Gradient Transforms and Wrappers

Method	Acts on	Extra pass?	Main effect	Typical use	Placement in stack
Norm clipping	Gradient norm	No	Bound exploding updates	General stability, recurrent or deep runs	Before optimizer step
AGC	Parameter-relative gradient norm	No	Scale clipping to parameter magnitude	NFNet-style or spiky conv blocks	Before optimizer step
Gradient centralization	Gradient tensor mean	No	Reduce gradient bias and smooth updates	Cheap universal stabilizer	Before state update
SAM	Loss landscape around θ	Yes, one extra backward	Favor flatter local neighborhoods	Generalization-focused runs	Around base optimizer
Lookahead	Fast-weight trajectory	No extra backward	Stabilize noisy optimizer paths	Wrap SGD, AdamW, or hybrids	Outside base optimizer
EMA / SWA	Iterates or checkpoints	No	Improve predictor smoothness	Noisy validation or late-training averaging	After optimizer step

Schedules and iterate averaging. The learning rate should be treated as an explicit schedule map

$$\eta_t = S(t; \lambda),$$

with warmup-plus-cosine as the sane default for AdamW-like training, and one-cycle or restart schedules available when the run length and batch regime make them attractive. Averaging lives at the end of the stack. EMA maintains

$$\bar{\theta}_t = \beta \bar{\theta}_{t-1} + (1 - \beta) \theta_t,$$

while SWA forms a late-training arithmetic average over iterates or checkpoints. Both change the effective predictor even when the inner optimizer is unchanged.[\[63, 68\]](#)

Sparse routing and MoE-specific optimization. For sparse experts, optimization is not only the base optimizer choice. Router stability is part of the loss geometry:

$$\mathcal{J}_t^{\text{MoE}} = \mathcal{L}_{\text{task}} + \lambda_{\text{lb}} L_{\text{load balance}} + \lambda_z L_z.$$

The router should usually live in its own parameter group so its learning rate, clipping, and regularization can be tuned independently of the experts and the shared trunk.[\[31\]](#)

Architecture-aware parameter grouping. The same learning rate and decay policy should not be applied blindly to every role. Norms and biases often want no decay. Routers often want smaller steps or extra clipping. Experts may want their own group. Matrix-heavy hidden layers may be delegated to Muon while the rest stays on AdamW. This is where the role tags introduced for initialization become useful again during descent.

Listing 12: Role-aware parameter bucketing for hybrid optimizers

```
def iter_named_parameters_with_tags(model: nn.Module):
    for module_name, module in model.named_modules():
        init_tag = getattr(module, "init_tag", None)
        for param_name, p in module.named_parameters(recurse=False):
            full_name = f"{module_name}.{param_name}" if module_name else param_name
            yield full_name, p, init_tag

def bucket_parameters(model: nn.Module):
    buckets = {
        "decay": [],
        "no_decay": [],
        "router": [],
        "expert": [],
        "head": [],
        "muon_2d": [],
    }

    for name, p, init_tag in iter_named_parameters_with_tags(model):
        if not p.requires_grad:
            continue

        role = None if init_tag is None else init_tag.role
```

```

if p.ndim == 1 or name.endswith("bias") or "norm" in name.lower():
    buckets["no_decay"].append((name, p))
elif role == "router":
    buckets["router"].append((name, p))
elif role == "expert":
    buckets["expert"].append((name, p))
elif role == "head":
    buckets["head"].append((name, p))
elif p.ndim == 2 and role not in {"router", "head"}:
    buckets["muon_2d"].append((name, p))
else:
    buckets["decay"].append((name, p))

return buckets

```

The main conceptual point is that gradient descent is a stack, not a single choice. The base optimizer, the preconditioner geometry, the trust ratio, the gradient transforms, the task-gradient aggregator, the router auxiliaries, and the averaging rule all act on the same update map. That is why this material belongs in the optimization spec rather than in ad hoc training-script folklore.

2.4 Hyperparameter Optimization

Hyperparameter optimization is the outer-loop problem of choosing λ when training is expensive, noisy, conditional, and budgeted. In a Machinator setting, the search space is almost always hierarchical because architecture choices activate or deactivate relevant hyperparameters. If sparse experts are present, router regularization and capacity terms appear. If multi-task heads are present, task-weighting and gate regularization appear. If fidelity is variable, epochs, steps, tokens, or data fraction become budget variables.[\[71\]](#)

The formal problem is

$$F(\lambda) = \mathcal{V}(\theta^*(\lambda), A), \quad \theta^*(\lambda) = \text{Train}(A, \lambda, b),$$

subject to constraints such as

$$c_j(\lambda) \leq 0 \quad \text{for latency, memory, cost, or throughput.}$$

Search-space structure. The important design fact is that the search space is conditional and mixed:

$$\lambda = (\lambda_{\text{disc}}, \lambda_{\text{cont}}, \lambda_{\text{cond}}, b),$$

with discrete choices, continuous scales, architecture-conditional knobs, and explicit budgets. That structure is why grid search is an anti-pattern and why methods that respect tree-structured or budgeted spaces matter most.

Practical method families. Random and Sobol sampling are the baseline because they expose bad regions of the space cheaply. TPE and tree-structured SMBO are excellent defaults for everyday conditional spaces. GP-based BO matters when trials are very expensive and the effective dimension is moderate. Hyperband and ASHA matter when partial learning curves are informative early.

BOHB and related hybrids combine smarter suggestions with aggressive pruning. PBT and PB2 matter when the quantity being tuned is really a *schedule* rather than a fixed scalar. Multi-objective or constrained HPO becomes mandatory once latency, memory, or cost is part of the target.[72, 73, 74, 75, 76, 77, 78]

Comparison Table: HPO Method Families

Family	Search type	Conditional spaces?	Multi-fidelity?	Best when	Core tradeoff
Random / Sobol	Sampling baseline	Yes	No	New search spaces or initial design points	Cheap and robust, not sample efficient
TPE / tree SMBO	Model based black-box	Yes	Limited	Mixed categorical and conditional knobs	Strong everyday default
GP Bayesian optimization	Surrogate + acquisition	Weak to moderate	No	Very expensive continuous trials	Sample efficient, weaker in ugly high dimension
ASHA / Hyperband	Budgeted racing	Indirectly	Yes	Early curves are informative	Fast culling, little modeling
BOHB / hybrids	Model based + racing	Yes	Yes	Need good anytime performance	More moving parts, better budget use
PBT / PB2	Online population search	Yes	Yes, via checkpoints	Schedules, not constants	Requires reliable checkpoint / resume
Constrained / multi-objective BO	Pareto or constrained search	Yes	Sometimes	Latency, memory, and cost matter directly	Better deployment fit, more bookkeeping

What the framework must expose. Every backend, regardless of sophistication, wants the same interface: a black-box trainable with conditional search space construction, intermediate metric reporting, budget control, checkpoint and resume, and separate reporting of constraints.[78, 79]

Listing 13: Conditional HPO space keyed from the architecture spec

```
def suggest_hparams(trial, arch_spec):
    h = {
        "optimizer": trial.suggest_categorical("optimizer", ["adamw", "sgd"]),
        "lr": trial.suggest_float("lr", 1e-5, 1e-1, log=True),
        "weight_decay": trial.suggest_float("weight_decay", 1e-6, 1e-1, log=True),
        "warmup_steps": trial.suggest_int("warmup_steps", 0, 4000),
    }

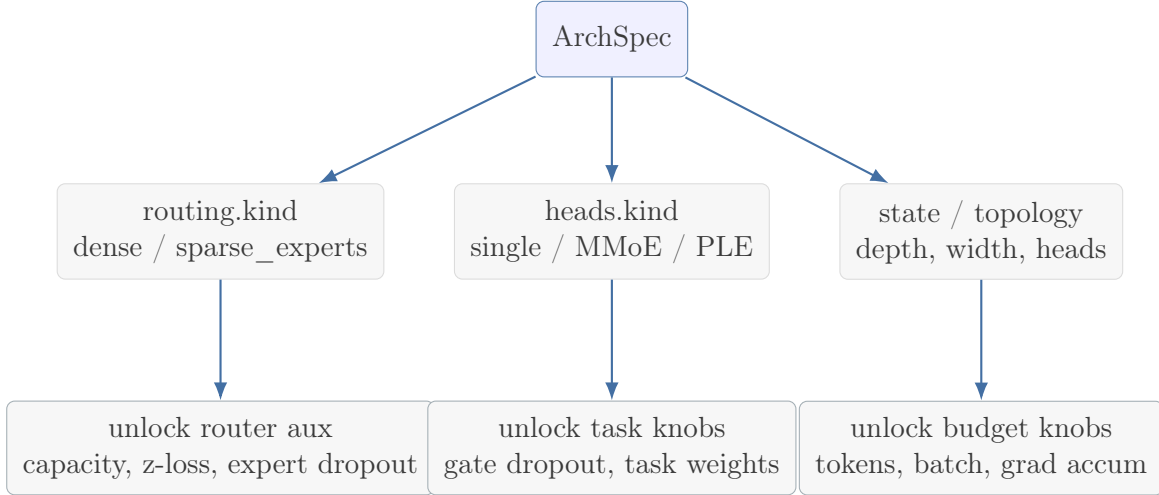
    if arch_spec["routing"]["kind"] == "sparse_experts":
        h["capacity_factor"] = trial.suggest_float("capacity_factor", 1.0, 2.0)
        h["router_aux_weight"] = trial.suggest_float("router_aux_weight", 1e-4, 1e-1, log=True)

    if arch_spec["heads"]["kind"] == "mmoe":
        h["gate_dropout"] = trial.suggest_float("gate_dropout", 0.0, 0.4)
```

```
return h
```

That conditional interface is the reason HPO belongs inside the spec system rather than outside it. The architecture registry tells the HPO layer which knobs exist, and the HPO layer in turn chooses training policies that determine whether the architecture ever reaches a useful regime.

Visual: Conditional Search Space Driven by the Architecture Spec



2.5 Architecture Optimization

Architecture optimization is the outer-loop problem in which the variables being optimized are not only scalar training knobs, but structural choices in the model itself. In Machinator terms, those choices live in the same places highlighted by Section 1:[\[80\]](#)

$$\gamma = (\gamma_{\text{relations}}, \gamma_{\text{blocks}}, \gamma_{\text{topology}}, \gamma_{\text{state}}, \gamma_{\text{routing}}, \gamma_{\text{mask}}).$$

These variables can select operators, activate or deactivate edges, choose widths or expert counts, alter sparsity masks, or change routing policies.

The clean mathematical statement is bilevel:

$$\gamma^* = \arg \min_{\gamma \in \mathcal{A}} \mathcal{V}(\theta^*(\gamma), \gamma), \quad \theta^*(\gamma) = \arg \min_{\theta} \mathcal{J}_{\text{train}}(\theta, \gamma).$$

Relaxed gradient search. DARTS-style methods relax discrete choices into trainable mixtures. A mutable site with candidate operators $\{o_1, \dots, o_M\}$ becomes

$$\tilde{o}(x) = \sum_{m=1}^M \text{softmax}(\gamma)_m o_m(x),$$

where γ are architecture logits. This is attractive because it reuses backpropagation directly, but it also introduces supernet bias and mixed-op interference.[\[81\]](#)

Sampled differentiable search. Single-path or Gumbel-style variants replace full soft mixing by sampled subgraphs or hard relaxations. This is often a better fit for relation choices, local block choices, and small topology toggles because the search objective can incorporate latency or memory more honestly.[82]

Controller, surrogate, and evolutionary search. When the search space is highly discrete, hardware-measured, or awkwardly structured, it is cleaner to externalize the search: sample a concrete config, materialize the model, evaluate it, and update a controller, surrogate, or mutation policy. In this regime the core requirements are weight inheritance, mutation operators over the spec, and a reliable evaluator.[83, 84]

Elastic supernets, growth, and sparsity. Cardinality variables such as width, depth, number of heads, number of slots, or number of experts are often handled more naturally by elastic supernets or growth policies than by soft operator mixtures. Dynamic sparsity and mask-based search fit the same pattern: the optimized object is a live mask or subnetwork, not a categorical op choice.[85, 88, 86, 87]

Comparison Table: Architecture Optimization Families

Family	Optimized object	Search mode	Best for	Strength	Tradeoff
Relaxed gradient search	Operator logits at mutable sites	Bilevel differentiable	Small op libraries, soft search spaces	Reuses backprop directly	Supernet bias and mixed-op interference
Sampled differentiable search	Single-path or hard samples	Stochastic differentiable	Hardware-aware op choice, cheaper search	Better memory / cost realism	Still weight-sharing biased
Controller / surrogate search	Discrete config dictionary	RL or model-based outer loop	Ugly conditional search spaces	Flexible and backend agnostic	More expensive evaluations
Evolutionary search	Mutations over the spec	Population black-box search	Hardware-measured or irregular spaces	Robust and simple to reason about	Many evaluations unless inheritance is strong
Elastic supernets	Width, depth, heads, slots, experts	Subnet sampling	Cardinality search under budgets	One supernet covers many children	Training the supernet is its own project
Growth / morphism	Topology edits over time	Online model transformation	Start-small then grow systems	Avoids paying full search cost up front	More orchestration complexity
Dynamic sparsity	Masks or live sparse graphs	Mask optimization / regrowth	Sparse parameters or sparse relations	Matches structured pruning goals	Harder export and hardware alignment

The implementation pattern should therefore center on mutable sites, an explicit search context, and a materialization step that exports a concrete architecture when search is over.

Listing 14: One mutable site for differentiable or sampled architecture search

```
@dataclass
class SearchContext:
    mode: str = "soft" # soft / gumbel / sample / fixed
```

```

temperature: float = 1.0
hard: bool = False
fixed_choices: dict[str, int] | None = None

class ChoiceOp(nn.Module):
    def __init__(self, name, ops):
        super().__init__()
        self.name = name
        self.names = list(ops.keys())
        self.ops = nn.ModuleList([ops[k] for k in self.names])
        self.logits = nn.Parameter(torch.zeros(len(self.names)))

    def forward(self, x, ctx: SearchContext):
        if ctx.mode == "fixed":
            idx = ctx.fixed_choices[self.name]
            y = self.ops[idx](x)
            weights = F.one_hot(torch.tensor(idx, device=x.device), num_classes=len(self.ops)).float()
            return y, {"choice": self.names[idx], "weights": weights}

        if ctx.mode == "gumbel":
            weights = F.gumbel_softmax(
                self.logits,
                tau=ctx.temperature,
                hard=ctx.hard,
                dim=-1,
            )

        elif ctx.mode == "sample":
            dist = Categorical(logits=self.logits / ctx.temperature)
            idx = dist.sample()
            weights = F.one_hot(idx, num_classes=len(self.ops)).float()

        else:
            weights = torch.softmax(self.logits / ctx.temperature, dim=-1)

        ys = [op(x) for op in self.ops]
        y = sum(weights[i] * ys[i] for i in range(len(ys)))
        return y, {"weights": weights, "op_names": self.names}

```

Once a framework has mutable relation blocks, mutable local blocks, cost oracles, and a materializer, the different NAS families stop looking like unrelated infrastructure projects. They become different outer optimizers over the same architecture state.

Optimization is therefore the natural partner to the architecture formalism from Section 1. The first section says that many model families can be reduced to a common typed routed program. This section says that initialization, descent, HPO, and architecture search can likewise be reduced to a common family of operators acting on that same program.

3 Objective Specifications

Architecture determines what may be computed. Optimization determines how parameters move. Objective specification determines what the system is asked to predict, how correctness is scored,

which auxiliary terms are active, which constraints are non-negotiable, and how checkpoints are finally selected. In Machinator, the objective layer is therefore not a single loss-function name. It is the semantic contract that binds heads, targets, metrics, and deployment criteria to the typed routed program already defined by the architecture and optimization layers.[20, 8, 37, 69, 70]

3.1 How Objective Families Fit Together

Let $\mathcal{K}$ denote the set of supervised or self-supervised tasks attached to a model. For each task $k \in \mathcal{K}$, the objective layer must specify:

$$\hat{y}_k = \rho_k(h_L(x; \theta, \gamma), x), \quad y_k \in \mathcal{Y}_k,$$

where $\mathcal{Y}_k$ is the target space and ρ_k is the readout contract from the final state field to the prediction space used by that task.

The objective specification can be written as

$$\Xi = (\mathcal{Y}, \rho, \ell, \omega, R, C, M, \Delta),$$

where $\mathcal{Y}$ is the family of target spaces and masks, ρ is the family of readout contracts, ℓ is the family of primary losses, ω is the task or sample weighting policy, R is the auxiliary-regularizer set, C is the constraint set, M is the metric family, and Δ is the checkpoint or model-selection rule. Objective families differ by how they instantiate this tuple, not by requiring unrelated training interfaces.

For masked minibatches, a task loss takes the generic form

$$\mathcal{L}_k = \frac{1}{Z_k} \sum_{i=1}^B m_{ik} \ell_k(\hat{y}_{ik}, y_{ik}), \quad Z_k = \sum_{i=1}^B m_{ik},$$

and the assembled training objective becomes

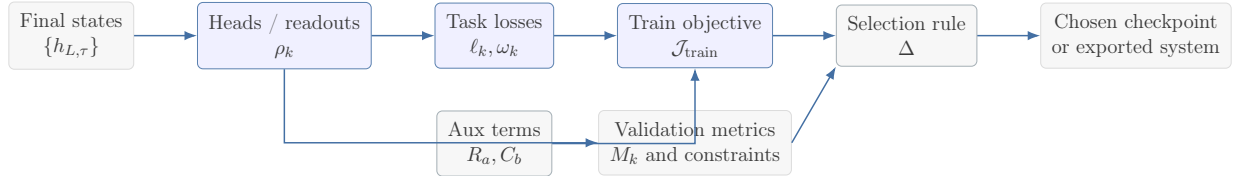
$$\mathcal{J}_{\text{train}} = \sum_{k \in \mathcal{K}} \omega_k \mathcal{L}_k + \sum_a \lambda_a R_a + \sum_b \mu_b \max(0, c_b).$$

This makes the abstraction boundary clear: the objective layer decides which supervised and auxiliary terms exist, while the optimization layer decides how the gradients from those terms are aggregated and used.

Comparison Table: Objective Family Summary

Family	Target space	Readout contract	Primary loss	Typical metrics	Common extras
Multiclass classification	Categorical label	Pooled or slot logits	Cross-entropy	Accuracy, F1, NLL	Label smoothing, calibration
Multilabel classification	Independent Bernoulli vector	Per-label logits	Binary cross-entropy	AUROC, mAP, F1	Threshold tuning, class weights
Regression	$\mathbb{R}^d$	Linear or bounded head	MSE, MAE, Huber, NLL	MAE, RMSE, R^2	Scale normalization, uncertainty head
Dense prediction	Per-token, per-pixel, or per-node field	Sitewise head over a structured index set	Pointwise CE or regression + masking	IoU, token F1, PSNR	Void masks, class imbalance weights
Set or query prediction	Finite set, matched queries, or boxes	Query-slot or set head	Matched set loss or detection loss	mAP, set accuracy, cardinality error	Hungarian matching, no-object terms
Retrieval / contrastive	Embedding space or ranked items	Projection head or scorer	Contrastive, pairwise, listwise loss	Recall@k, MRR, NDCG	Temperature, hard-negative mining
Multi-task / constrained	Product of several task spaces	One head per task or shared expert heads	Weighted sum of task losses	Task metrics + aggregate score	Router auxiliaries, latency or cost constraints

Visual: Objective Assembly and Selection



3.2 Target Spaces and Readout Contracts

The first objective decision is not the loss; it is the prediction contract. A token classifier supervises token states. A graph classifier supervises a pooled global state. A query detector supervises learned query slots. A multi-task system may supervise several state types at once.[20, 8, 37, 39, 40] In formal terms, each task installs a map

$$\rho_k : \{h_{L,\tau}\}_{\tau \in \mathcal{T}} \times x \rightarrow \hat{\mathcal{Y}}_k,$$

and the objective layer must say which state types the task is allowed to read, whether masking is active, and whether the output is pooled, sitewise, querywise, or set structured.

This is exactly where architecture and objective specifications meet. The architecture layer guarantees that certain state fields, heads, and routing paths exist. The objective layer decides which of those states are meaningful prediction sites and what the semantic type of each prediction should be. Two models may share the same backbone and optimization policy yet differ completely in objective specification because they read different states, predict into different target spaces, or use different model-selection criteria.

3.3 Loss Assembly, Regularizers, and Auxiliary Terms

Once the target space is fixed, the primary loss family should match its geometry. Classification wants probabilistic or margin-style losses. Regression wants scale-sensitive or robust losses. Dense prediction wants sitewise masking. Set prediction wants matching-aware supervision. Retrieval wants pairwise or contrastive objectives. The point of the objective layer is not to enumerate every named loss, but to expose the small number of semantic choices they represent.

Auxiliary terms belong in the same section because they encode additional desiderata that are not identical to the primary prediction error. Router load-balancing and z -loss terms for sparse experts are objective-level statements about how the system should behave, not optimizer tricks.[31] The same is true for consistency losses, sparsity penalties, equivariance penalties, or calibration regularizers. Once these terms are explicit, the optimization layer can decide whether their gradients are simply summed, uncertainty weighted, or fed through conflict-aware aggregation rules.[69, 70]

3.4 Metrics, Constraints, and Model Selection

Training loss and validation success are usually not the same object. A model may be trained with cross-entropy but selected by macro-F1, trained with Huber loss but selected by MAE, or trained under a penalized objective while still requiring hard deployment constraints on latency or memory. The objective layer therefore has to expose a separate evaluation contract:

$$M_k^{\text{val}} = M_k(\{\hat{y}_{ik}, y_{ik}\}_{i=1}^{N_{\text{val}}}), \quad c_j^{\text{val}} \leq 0.$$

Checkpoint selection is then a decision rule rather than a raw metric:

$$t^* = \Delta(\{M_k^{\text{val}}(t)\}, \{c_j^{\text{val}}(t)\}),$$

where Δ may be a single-metric argmax, a constrained argmax, a lexicographic rule, or a Pareto filter. This is the same contract that downstream HPO and NAS loops should query, which is why objective specification cannot be reduced to a loss-function line buried in a training script.[71]

3.5 Compact Objective View

The compressed ‘ObjectiveSpec’ used earlier is enough for the optimization registry, but it helps to also show the richer semantic view it is standing in for:

Listing 15: Expanded objective view compatible with the registry

```
OBJECTIVE_SPEC_VIEW = {
  "tasks": {
    "intent": {
      "head": "cls_token",
      "target_space": "categorical[48]",
      "loss": "cross_entropy",
      "metrics": ["accuracy", "macro_f1"],
      "weight": 1.0,
    },
    "value": {
      "head": "pooled_regression",
```

```

        "target_space": "real[1]",
        "loss": "huber",
        "metrics": ["mae", "rmse"],
        "weight": 0.25,
    },
},
"regularizers": {
    "label_smoothing": 0.1,
    "router_aux_weight": 1e-2,
},
"constraints": {
    "max_latency_ms": 15.0,
    "max_params_m": 80.0,
},
"selection": {
    "primary_metric": "val/macro_f1",
    "direction": "max",
    "must_satisfy": ["latency_ms <= 15.0"],
    "tie_break": "val/nll",
},
}

```

If architecture answers where computation lives and optimization answers how it moves, objective specification answers what the computation is for. That is why targets, losses, metrics, constraints, and selection rules belong inside the same formal outline rather than in separate notes or trainer-specific conventions.

References

References

- [1] Battaglia et al., “Relational inductive biases, deep learning, and graph networks,” <https://arxiv.org/abs/1806.01261>.
- [2] Battaglia et al., “Relational inductive biases, deep learning, and graph networks” (graph-network formalism reference), <https://arxiv.org/abs/1806.01242>.
- [3] Zaheer et al., “Deep Sets,” <https://arxiv.org/abs/1703.06114>.
- [4] Lee et al., “Set Transformer,” <https://arxiv.org/abs/1810.00825>.
- [5] Dosovitskiy et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” <https://arxiv.org/abs/2010.11929>.
- [6] Locatello et al., “Object-Centric Learning with Slot Attention,” <https://arxiv.org/abs/2006.15055>.
- [7] Jaegle et al., “Perceiver IO,” <https://arxiv.org/abs/2103.03206>.
- [8] Carion et al., “DETR,” <https://arxiv.org/abs/2005.12872>.
- [9] He et al., “Deep Residual Learning for Image Recognition,” <https://arxiv.org/abs/1512.03385>.
- [10] Srivastava et al., “Highway Networks,” <https://arxiv.org/abs/1505.00387>.
- [11] Huang et al., “DenseNet,” <https://arxiv.org/abs/1608.06993>.
- [12] Ronneberger et al., “U-Net,” <https://arxiv.org/abs/1505.04597>.
- [13] Lin et al., “Feature Pyramid Networks for Object Detection,” <https://arxiv.org/abs/1612.03144>.
- [14] Simonyan and Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” <https://arxiv.org/abs/1409.1556>.
- [15] van den Oord et al., “WaveNet,” <https://arxiv.org/abs/1609.03499>.
- [16] Chollet, “Xception,” <https://arxiv.org/abs/1610.02357>.
- [17] Hu et al., “Squeeze-and-Excitation Networks,” <https://arxiv.org/abs/1709.01507>.
- [18] Liu et al., “Swin Transformer,” <https://arxiv.org/abs/2103.14030>.
- [19] Liu et al., “ConvNeXt,” <https://arxiv.org/abs/2103.15808>.
- [20] Vaswani et al., “Attention Is All You Need,” <https://arxiv.org/abs/1706.03762>.
- [21] Shaw et al., “Self-Attention with Relative Position Representations,” <https://arxiv.org/abs/1803.02155>.
- [22] Su et al., “RoFormer / Rotary Position Embedding,” <https://arxiv.org/abs/2104.09864>.
- [23] Press et al., “ALiBi,” <https://arxiv.org/abs/2108.12409>.
- [24] Gilmer et al., “Neural Message Passing for Quantum Chemistry,” <https://arxiv.org/abs/1704.01212>.
- [25] Kipf and Welling, “Semi-Supervised Classification with Graph Convolutional Networks,” <https://arxiv.org/abs/1609.02907>.
- [26] Velickovic et al., “Graph Attention Networks,” <https://arxiv.org/abs/1710.10903>.
- [27] Hu et al., “Heterogeneous Graph Transformer,” <https://arxiv.org/abs/2003.01332>.
- [28] Chien et al., “You are AllSet: A Multiset Function Framework for Hypergraph Neural Networks,” <https://arxiv.org/abs/2106.13264>.

- [29] Shazeer et al., “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer,” <https://arxiv.org/abs/1701.06538>.
- [30] Lepikhin et al., “GShard,” <https://arxiv.org/abs/2006.16668>.
- [31] Fedus et al., “Switch Transformers,” <https://arxiv.org/abs/2101.03961>.
- [32] Ha et al., “HyperNetworks,” <https://arxiv.org/abs/1609.09106>.
- [33] Houshy et al., “Parameter-Efficient Transfer Learning for NLP,” <https://arxiv.org/abs/1902.00751>.
- [34] Cohen and Welling, “Group Equivariant Convolutional Networks,” <https://arxiv.org/abs/1602.07576>.
- [35] Finzi et al., “Generalizing Convolutional Neural Networks for Equivariance to Lie Groups on Arbitrary Continuous Data,” <https://proceedings.mlr.press/v119/finzi20a/finzi20a.pdf>.
- [36] Satorras et al., “E(n) Equivariant Graph Neural Networks,” <https://arxiv.org/abs/2102.09844>.
- [37] Luong et al., “Multi-task Sequence to Sequence Learning,” <https://arxiv.org/abs/1511.06114>.
- [38] Misra et al., “Cross-Stitch Networks,” <https://arxiv.org/abs/1604.03539>.
- [39] Ma et al., “Modeling Task Relationships in Multi-task Learning with Multi-gate Mixture-of-Experts,” <https://dl.acm.org/doi/10.1145/3219819.3220007>.
- [40] Tang et al., “Progressive Layered Extraction for Multi-Task Learning,” <https://dl.acm.org/doi/10.1145/3383313.3412236>.
- [41] Glorot and Bengio, “Understanding the difficulty of training deep feedforward neural networks,” <https://proceedings.mlr.press/v9/glorot10a/glorot10a.pdf>.
- [42] Zhang et al., “Fixup Initialization,” <https://arxiv.org/pdf/1901.09321>.
- [43] Wang et al., “DeepNet / DeepNorm,” <https://arxiv.org/pdf/2203.00555>.
- [44] Mishkin and Matas, “All You Need is a Good Init (LSUV),” <https://arxiv.org/abs/1511.06422>.
- [45] Xiong et al., “On Layer Normalization in the Transformer Architecture,” <https://arxiv.org/abs/2002.04745>.
- [46] Yang et al., “Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer,” <https://arxiv.org/abs/2203.03466>.
- [47] Tallec and Ollivier, “Can Recurrent Neural Networks Warp Time?” <https://research.google.com/pubs/archive/44961.pdf>.
- [48] Li et al., “On the Initialization of Graph Neural Networks,” <https://arxiv.org/abs/2312.02622>.
- [49] Meng et al., “PiSSA,” <https://arxiv.org/abs/2404.02948>.
- [50] Bachlechner et al., “ReZero,” <https://arxiv.org/pdf/2002.10444>.
- [51] Huang et al., “T-Fixup,” <https://proceedings.mlr.press/v119/huang20f/huang20f.pdf>.
- [52] PyTorch optimizer documentation, <https://docs.pytorch.org/docs/stable/optim.html>.
- [53] Sutskever et al., “On the importance of initialization and momentum in deep learning,” <https://proceedings.mlr.press/v28/sutskever13.html>.
- [54] Kingma and Ba, “Adam,” <https://arxiv.org/abs/1412.6980>.
- [55] Loshchilov and Hutter, “Decoupled Weight Decay Regularization,” <https://arxiv.org/abs/1711.05101>.
- [56] Shazeer and Stern, “Adafactor,” <https://arxiv.org/abs/1804.04235>.
- [57] PyTorch Muon documentation, <https://docs.pytorch.org/docs/stable/generated/torch.optim.Muon.html>.

- [58] You et al., “Large Batch Training of Convolutional Networks with LARS,” <https://arxiv.org/abs/1708.03888>.
- [59] You et al., “Large Batch Optimization for Deep Learning: Training BERT in 76 minutes,” <https://arxiv.org/abs/1904.00962>.
- [60] Martens and Grosse, “Optimizing Neural Networks with Kronecker-factored Approximate Curvature,” <https://arxiv.org/abs/1503.05671>.
- [61] Gupta et al., “Shampoo,” <https://arxiv.org/abs/1802.09568>.
- [62] Vyas et al., “SOAP: Improving and Stabilizing Shampoo using Adam,” <https://arxiv.org/abs/2409.11321>.
- [63] Liu et al., “On the Variance of the Adaptive Learning Rate and Beyond,” <https://arxiv.org/abs/1910.04209>.
- [64] Pascanu et al., “On the difficulty of training recurrent neural networks,” <https://arxiv.org/abs/1211.5063>.
- [65] Brock et al., “High-Performance Large-Scale Image Recognition Without Normalization,” <https://arxiv.org/abs/2102.06171>.
- [66] Zhang et al., “Lookahead Optimizer,” <https://arxiv.org/abs/1907.08610>.
- [67] Foret et al., “Sharpness-Aware Minimization,” <https://arxiv.org/abs/2010.01412>.
- [68] Izmailov et al., “Averaging Weights Leads to Wider Optima and Better Generalization,” <https://arxiv.org/abs/1803.05407>.
- [69] Kendall et al., “Multi-Task Learning Using Uncertainty to Weigh Losses,” <https://arxiv.org/abs/1705.07115>.
- [70] Sener and Koltun, “Multi-Task Learning as Multi-Objective Optimization,” <https://arxiv.org/abs/1810.04650>.
- [71] Bischl et al., “Hyperparameter Optimization: Foundations, Algorithms, Best Practices, and Open Challenges,” <https://wires.onlinelibrary.wiley.com/doi/10.1002/widm.1484>.
- [72] Bergstra and Bengio, “Random Search for Hyper-Parameter Optimization,” <https://jmlr.org/papers/v13/bergstra12a.html>.
- [73] Bergstra et al., “Algorithms for Hyper-Parameter Optimization,” <https://papers.neurips.cc/paper/4443-algorithms-for-hyper-parameter-optimization.pdf>.
- [74] Snoek et al., “Practical Bayesian Optimization of Machine Learning Algorithms,” <https://arxiv.org/abs/1206.2944>.
- [75] Li et al., “Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization,” <https://jmlr.org/papers/v18/16-558.html>.
- [76] Falkner et al., “BOHB,” <https://arxiv.org/abs/1807.01774>.
- [77] Jaderberg et al., “Population Based Training,” <https://arxiv.org/abs/1711.09846>.
- [78] Optuna documentation, “Efficient optimization algorithms,” https://optuna.readthedocs.io/en/stable/tutorial/10_key_features/003_efficient_optimization_algorithms.html.
- [79] Ray Tune trainable API documentation, <https://docs.ray.io/en/latest/tune/api/trainable.html>.
- [80] White et al., “Neural Architecture Search: Insights from 1000 Papers,” <https://arxiv.org/abs/2301.08727>.
- [81] Liu et al., “DARTS,” <https://arxiv.org/abs/1806.09055>.

- [82] Xie et al., “SNAS,” <https://arxiv.org/abs/1812.09926>.
- [83] Zoph and Le, “Neural Architecture Search with Reinforcement Learning,” <https://arxiv.org/abs/1611.01578>.
- [84] Real et al., “Large-Scale Evolution of Image Classifiers,” <https://arxiv.org/abs/1703.01041>.
- [85] Yu et al., “Slimmable Neural Networks,” <https://arxiv.org/abs/1812.08928>.
- [86] Chen et al., “Net2Net,” <https://arxiv.org/abs/1511.05641>.
- [87] Mocanu et al., “Scalable Training of Artificial Neural Networks with Adaptive Sparse Connectivity inspired by Network Science,” <https://arxiv.org/abs/1707.04780>.
- [88] Cai et al., “Once-for-All,” <https://arxiv.org/abs/1908.09791>.